

Codificación de módulos del software.

GA7-220501096-AA2-EV01.

Fase:

GA7

Aprendiz

JEANNY PATRICIA GARCÍA M

Instructor

DAVID FERNANDO GUERRERO VANEGAS

Proyecto Java Versión 1.0

SERVICIO NACIONAL DE APRENDIZAJE (SENA)

ANÁLISIS Y DESARROLLO DE SOFTWARE – FICHA #3186636

2026

CONTENIDO

1. Introducción

- a. Objetivos de la evidencia

2. Lineamientos

3. Marco Teórico

- a. **JDBC** (Java Database Connectivity): ¿Qué es?, requisitos de uso y componentes principales (DriverManager, Connection, Statement, PreparedStatement, ResultSet y Método next()).
- b. **Operaciones CRUD**: Definición y acciones básicas (Create/INSERT, Read/SELECT, Update/UPDATE, Delete/DELETE).
- c. **Hibernate**: Concepto de ORM, Entidades, Anotaciones básicas (@Id, @GeneratedValue, @Column), SessionFactory, Session y Ciclo de vida de una entidad (Transient, Persistent, Detached).
- d. **Relaciones en Base de Datos**: Mapeos OneToOne, OneToMany y ManyToMany.

4. Actividad

- a. **Introducción al Proyecto**: Funcionalidades del módulo, arquitectura, Modelo ER y estructura en la base de datos MySQL.
- b. **Fase 1: Configuración y Modelado (Pasos 1 al 4)**: Inicialización del proyecto (Spring Initializr), configuración de application.properties, creación de la entidad (ProductoEntity), controlador inicial (HomeController) y repositorio (ProductoRepository).
- c. **Fase 2: Lógica de Negocio y Servicios (Pasos 5 al 10)**: Creación de ProductoService e implementación lógica de todas las operaciones CRUD (Guardar, Obtener todos, Buscar por ID, Actualizar y Eliminar).
- d. **Fase 3: Controlador y Vista Principal (Pasos 11 al 15)**: Construcción de ProductoController, visualización del listado en index.html (Card dinámica), ejecución de INSERTS de prueba, integración de imágenes estáticas y configuración del Hero.
- e. **Fase 4: Formularios de Creación (Pasos 16 al 19)**: Maquetación de la vista nuevo.html, creación de las rutas (GET/POST) para mostrar el formulario, método definitivo para guardar productos (incluyendo archivos de imagen) y validación de datos.
- f. **Fase 5: Edición, Eliminación y Detalle (Pasos 20 al 22)**: Implementación de parámetros de URL (@PathVariable) para las funciones de editarProducto(), eliminarProducto() y la creación de la vista individual detalle.html.

5. Git

6. Estándares de Codificación

7. Conclusiones

INTRODUCCION

El desarrollo de aplicaciones que interactúen con la base de datos es uno de los pilares del desarrollo de software. En Java, la tecnología standard para establecer esa conexión es JDBC o Java Database Connectivity, una API que permite conectar aplicaciones Java con sistemas gestores de base de datos relacionales, ejecutar instrucciones en SQL y manipular la información almacenada de manera segura y eficiente.

En esta evidencia se implementa un módulo del proyecto **Footsy**, utilizando Hibernate para realizar las operaciones básicas de persistencia de datos (CRUD) sobre la entidad **Producto**, siguiendo los estándares de codificación, el uso de herramientas de versionamiento y las buenas prácticas de desarrollo de software.

Objetivo de la evidencia

Desarrollar el módulo Producto del proyecto Footsy utilizando Hibernate, implementando las operaciones CRUD sobre una base de datos MySQL, siguiendo los estándares de codificación y utilizando Git como herramienta de control de versiones.

LINEAMIENTOS

Teniendo en cuenta las características del software a desarrollar, realizar la codificación del módulo del proyecto, realizando conexiones con bases de datos por medio de JDBC, tomando como referencia lo visto en el componente formativo.

Elementos para tener en cuenta: o Para la codificación del módulo debe tener en cuenta los artefactos del ciclo del software realizados con anterioridad: diagrama de clases, diagramas de casos de uso, historias de usuario, diseños, prototipos, informe técnico de plan de trabajo para construcción de software con tecnologías seleccionadas, etc.

- Se debe crear el proyecto utilizando herramientas de versionamiento.
- El código debe cumplir con estándares de codificación, como:
 - Nombramiento de variables.
 - Nombramiento de métodos.
 - Nombramiento de clases.
 - Nombramiento de paquetes.
- Debe tener funcionalidades de inserción, consulta, actualización y eliminación.

Lineamientos generales para la entrega de la evidencia:

- Productos a entregar: carpeta comprimida que debe tener los siguientes archivos:
 - Archivos del proyecto.
 - Archivo con enlace del repositorio, la carpeta comprimida debe tener el nombre del aprendiz y número de la evidencia, así: NOMBREAPELLIDO_AA2_EV01. o Extensión: ZIP, RAR.
 - Para hacer el envío de la evidencia, remítase al área de la actividad correspondiente y acceda al espacio: Codificación de módulos del software. GA7-220501096-AA2-EV01.

JDBC

¿Qué es JDBC?

JDBC (Java Database Connectivity) es la tecnología que utiliza Java para comunicarse con una base de datos relacional. En realidad, Java por sí solo no sabe cómo conectarse a una base de datos, por eso necesita JDBC.

Podemos entender JDBC como un puente o un traductor entre Java y la base de datos. Gracias a JDBC, Java puede enviar instrucciones SQL, como consultar, insertar, modificar o eliminar datos, y también recibir la respuesta que devuelve la base de datos.

¿Qué se necesita para utilizar JDBC?

Para trabajar con JDBC necesitamos dos cosas principales:

Una base de datos (por ejemplo MySQL, PostgreSQL, Oracle o SQL Server).

El driver JDBC correspondiente a esa base de datos, ya que cada gestor tiene su propio controlador.

Además, durante la conexión intervienen CINCO componentes principales:

- DriverManager
- Connection
- Statement
- PreparedStatement
- ResultSet

DriverManager

El DriverManager es el encargado de localizar el driver adecuado según la base de datos que estamos utilizando.

Cuando Java solicita una conexión, el DriverManager identifica el driver correspondiente mediante la URL de conexión y utiliza ese driver para establecer la comunicación con la base de datos.

En otras palabras, es quien encuentra el "traductor" correcto para poder iniciar la conexión

Connection

La Connection representa la conexión activa entre la aplicación Java y la base de datos.

Una vez que el DriverManager encuentra el driver correcto y la conexión se establece correctamente, se obtiene un objeto Connection, que será el canal por el cual Java enviará todas las consultas SQL.

Statement

El Statement es el objeto encargado de enviar instrucciones SQL a la base de datos.

Con él podemos ejecutar consultas como:

- SELECT
- INSERT
- UPDATE
- DELETE

Es decir, toma las instrucciones SQL escritas en Java y las envía al gestor de la base de datos para que sean ejecutadas.

PreparedStatement

El PreparedStatement es una versión más segura y eficiente del objeto Statement. Permite ejecutar instrucciones SQL parametrizadas, es decir, consultas que reciben valores mediante parámetros en lugar de construir la sentencia concatenando texto.

Además de hacer el código más organizado, ayuda a prevenir ataques de inyección SQL y mejora el rendimiento cuando una misma consulta se ejecuta varias veces con diferentes valores.

En este proyecto se utilizará PreparedStatement para implementar todas las operaciones CRUD del módulo Producto.

ResultSet

El ResultSet almacena los resultados que devuelve una consulta SQL.

Por ejemplo, cuando ejecutamos un SELECT, la base de datos responde con los registros encontrados y esos datos llegan dentro de un objeto ResultSet, desde donde Java puede leerlos uno por uno.

Método next()

El método next() sirve para recorrer un ResultSet.

Cada vez que se ejecuta, el cursor avanza al siguiente registro. Mientras existan filas disponibles, el método devuelve true; cuando ya no quedan registros, devuelve false.

Por eso normalmente se utiliza dentro de un ciclo while, para leer todos los resultados obtenidos por la consulta.

Operaciones crud

Que son las Operaciones CRUD?

Las operaciones CRUD representan las cuatro acciones básicas que una aplicación realiza sobre la información almacenada en una base de datos. En el proyecto Footsy, estas operaciones permiten gestionar el módulo Producto, utilizando las sentencias SQL INSERT, SELECT, UPDATE y DELETE.

Create (INSERT)

La operación **Create** permite agregar un nuevo producto a la base de datos. Para ello se utiliza la sentencia INSERT, encargada de almacenar la información del producto, como su nombre, descripción, precio y stock, dentro de la tabla correspondiente.

- Por ejemplo, registrar un nuevo par de zapatillas en el catálogo de Footsy.

Read (SELECT)

La operación **Read** se utiliza para consultar y visualizar la información almacenada en la base de datos. Esta acción se realiza mediante la sentencia SELECT, que recupera uno o varios registros para mostrarlos al usuario.

Por ejemplo, visualizar el catálogo de productos disponibles o consultar un producto específico mediante su identificador.

Update (UPDATE)

La operación **Update** permite modificar la información de un producto que ya existe en la base de datos. Para ello se utiliza la sentencia UPDATE, que actualiza los campos necesarios sin crear un nuevo registro.

Por ejemplo, modificar el precio, la descripción o la cantidad disponible en stock de un producto.

Delete (DELETE)

La operación **Delete** permite eliminar productos que ya no forman parte del catálogo de la tienda. Esta acción se realiza mediante la sentencia DELETE, que elimina uno o varios registros de la base de datos según la condición establecida.

Por ejemplo, eliminar un producto que ha sido descontinuado o que ya no se encuentra disponible para la venta.

Hibernate

Que es Hibernate?

Hibernate es un software libre con licencia GNU LGPL que presta servicios de ORM para Java. Es un plugin que puede ser integrado al IDE de desarrollo en Java y facilita el proceso de mapeo de tablas en una base de datos relacional a objetos Java mediante el uso de archivos declarativos en formato XML o anotaciones.

Sin Hibernate, Java no sabe guardar objetos en una base de datos. La base de datos entiende de tablas, filas, columnas mientras que Java entiende de Objetos, Clases y atributos. Con JDBC puro se tendría que escribir mucho código para convertir un objeto en SQL. Hibernate nacio para resolver ese problema

Hibernate = ORM (Object Relational Mapping)

Hibernate actua como un traductor para dialogar con JAVA y la BASE DE DATOS

Objeto Java -> Tablas SQL y viceversa.

Entidades

Una entidad representa una tabla, colocando **@Entity** encima de la clase le decimos a Hibernate “esta clase representa una tabla de la base de datos”.

```
@Entity
public class ProductoEntity {
```

Id

La anotación **@Id** dice a hibernate que ese atributo es la clave primaria de la entidad.

GeneratedValue

La anotación `@GeneratedValue` le indica a Hibernate que nosotros no asignaremos manualmente el valor de la clave primaria, sino que será la base de datos quien lo genere automáticamente al momento de insertar el registro.

`@GeneratedValue(strategy = GenerationType.IDENTITY)` Indica a Hibernate utilizara la estrategia IDENTITY, dejando que la base de datos genere el valor del ID mediante una columna de autoincremento (AUTO_INCREMENT) en Mysql.

```
public class CategoriaEntity {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Integer id;
```

Column

La anotación `@Column` se utiliza para personalizar el mapeo de un atributo de la entidad con una columna de la base de datos.

Si no utilizamos `@Column`, Hibernate crea el mapeo automáticamente usando el nombre del atributo. Sin embargo, con `@Column` podemos indicar el nombre de la columna y configurar propiedades como si acepta valores nulos, su longitud, si es única, entre otras.

```
@Column(  
    name = "nombre_producto",  
    nullable = false,  
    length = 100,  
    unique = true  
)  
private String nombre;
```

name → Nombre de la columna en la base de datos.

nullable → Indica si la columna puede almacenar valores NULL.

length → Longitud máxima para columnas de tipo String.

unique → Indica que los valores de esa columna deben ser únicos.

insertable → Permite o no insertar valores en esa columna.

updatable → Permite o no actualizar esa columna.

SessionFactory

Llamada Fabrica de sesiones. Cuando inicia la aplicación crea una sola vez una SessionFactory al interior de esta guarda la configuración de la base de datos, las entidades, como conectarse y como crear sesiones.

Como su nombre lo dice su función es preparar el entorno y crear las sesiones que hagan falta.

Session

La session representa una unidad de trabajo, es la que administra y gestiona entidades, puede consultar la base de datos, guardar cambios y es la que contiene el contexto de persistencia de la sesión, cuando se cierra deja de administrar las entidades.

Aquí nacen conceptos importantes:

- Guardar nuevas entidades (persist()).
- Actualizar entidades (merge()).
- Eliminar entidades (remove()).
- Sincronizar los cambios con la base de datos (flush()).

Además, la Session contiene el contexto de persistencia (**Persistence Context**), que es un espacio de memoria donde Hibernate mantiene un seguimiento de todas las entidades que están siendo administradas. Mientras una entidad pertenece a este contexto, Hibernate detecta automáticamente cualquier modificación realizada sobre ella y la sincroniza con la base de datos cuando corresponde.

Cuando la Session se cierra (close()), el contexto de persistencia desaparece y las entidades dejan de estar administradas por Hibernate, pasando al estado Detached.

El ciclo de vida de una session es muy simple:

- Se crea a través de una openSession()
- Realiza su trabajo administrando o ejecutando operaciones en el DB
- Se cierra Close() liberando recursos y finalizando el contexto de persistencia.

Una vez está cerrada la sesión, hibernate deja de gestionar las entidades que pertenecían a ella y cualquier modificación posterior ya no podrá ser detectada.

Ciclo de vida de una entidad

Mientras una entidad se encuentra en estado **TRANSIENT**, Hibernate no sabe que existe. Para Hibernate es simplemente un objeto Java (POJO) que aún no forma parte del contexto de persistencia.

Cuando la entidad pasa al estado **PERSISTENT**, comienza a ser administrada por la Session. A partir de ese momento, Hibernate la

incorpora al Persistence Context y empieza a realizar el proceso conocido como Dirty Checking.

El **Dirty Checking** es un mecanismo mediante el cual Hibernate supervisa automáticamente todas las entidades persistentes para detectar si alguno de sus atributos ha sido modificado durante la sesión.

Hibernate no ejecuta un UPDATE cada vez que cambia un atributo. En su lugar, registra internamente los cambios y espera hasta el momento adecuado (normalmente durante un flush() o un commit()) para compararlos con el estado original de la entidad. Si detecta diferencias, genera un único UPDATE con el estado final de la entidad, evitando realizar múltiples actualizaciones y optimizando el acceso a la base de datos.

Cuando la Session se cierra mediante close(), la entidad pasa al estado **DETACHED**, lo que significa que ya no pertenece a ninguna Session y ha dejado de ser administrada por Hibernate.

RELACIONES

OneToOne (Uno a Uno)

Una fila de una tabla está relacionada con una sola fila de otra.

En java:

```
@OneToOne
@JoinColumn(name="pasaporte_id")
private Pasaporte pasaporte;
```

En la otra clase:

```
@OneToOne(mappedBy="pasaporte")
private Persona persona;
```

@JoinColumn le dice a Hibernate dónde está la llave foránea (Foreign Key), es decir, qué entidad es la propietaria de la relación.

mappedBy le dice a Hibernate que esa entidad no administra la relación; simplemente refleja la relación que ya está definida en la otra entidad.

OneToMany (Uno a Muchos)

Una fila de una tabla puede estar relacionada con muchas filas de otra tabla.

Ejemplo: Una categoría tiene muchos productos.

En Java (Categoría):

```
@OneToMany(mappedBy = "categoria")
private List<Producto> productos;
```

En la otra clase (Producto):

```
@ManyToOne
@JoinColumn(name = "categoria_id")
private Categoria categoria;
```

@ManyToOne + @JoinColumn le dicen a Hibernate que la llave foránea (categoria_id) está en la tabla Producto, por lo tanto Producto es el propietario de la relación.

mappedBy le dice a Hibernate que Categoría solo refleja la relación y que no debe crear otra llave foránea.

ManyToMany (Muchos a Muchos)

Muchas filas de una tabla pueden relacionarse con muchas filas de otra tabla.

Ejemplo: Un alumno puede inscribirse en muchos cursos y un curso puede tener muchos alumnos.

En Java (Alumno):

```
@ManyToMany
@JoinTable(
    name = "alumno_curso",
    joinColumns = @JoinColumn(name = "alumno_id"),
    inverseJoinColumns = @JoinColumn(name = "curso_id"))
private List<Curso> cursos;
```

En la otra clase (Curso):

```
@ManyToMany(mappedBy = "cursos")
private List<Alumno> alumnos;
```

@JoinTable le dice a Hibernate que la relación se guarda en una tabla intermedia (alumno_curso), ya que ninguna de las dos tablas puede contener una única llave foránea.

joinColumns indica la llave foránea de la entidad propietaria (alumno_id).

inverseJoinColumns indica la llave foránea de la otra entidad (curso_id).

mappedBy le dice a Hibernate que esa entidad solo refleja la relación y que la tabla intermedia ya está definida en la otra entidad.

ACTIVIDAD

El objetivo de esta evidencia es desarrollar el módulo producto del proyecto Footsy utilizando JAVA y JDBC, Implementando las operaciones CRUD sobre una base de datos MYSQL, siguiendo los estándares de codificación y utilizando GIT como herramienta de control de versiones.

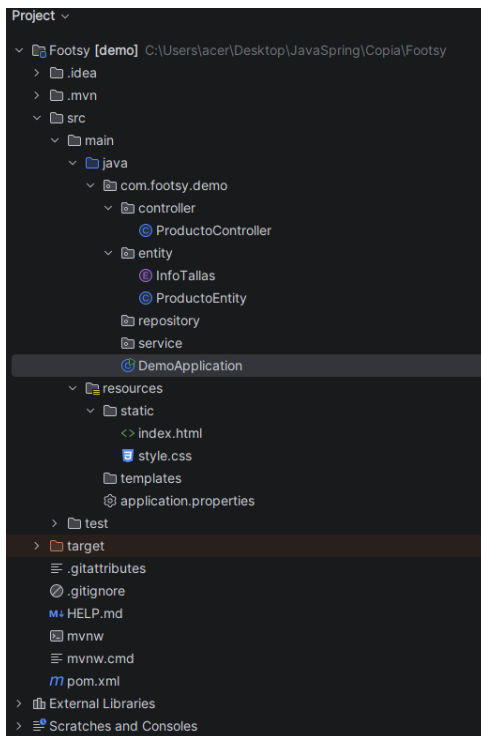
Funcionalidades del módulo Producto

La sección Producto debe permitir:

- Registrar un producto.
- Consultar todos los productos.
- Buscar un producto por su ID.
- Actualizar la información de un producto.
- Eliminar un producto.

Arquitectura del proyecto

En esta primera actividad se implementará únicamente el módulo **Producto**, ya que será la base del desarrollo del proyecto. Sin embargo, también se han definido otras entidades como Categoría, Imagen e Inventario, las cuales se ampliarán en las siguientes evidencias a medida que el profesor vaya desarrollando estos temas en clase.



Modelo ER

Para esta actividad se trabajará únicamente con la tabla Producto

PRODUCTO	
id	INTEGER PK
categoria_id	INTEGER FK
esEstadoActivo	BOOLEAN
fechaCreacion	TIMESTAMP
fechaActualizacion	TIMESTAMP
codigo	VARCHAR UNIQUE
nombre	VARCHAR
descripcion	VARCHAR
precioUnitario	DOUBLE
precioVenta	DOUBLE
infoTallas	VARCHAR
peso	VARCHAR
dimensiones	VARCHAR

Base de datos en Mysql (db_footsy_java)(Producto)

MySQL Workbench

root x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- contact
- contactanos
- db_footsy
- db_footsy_java
 - Tables
 - producto_entity
 - Columns
 - id
 - codigo
 - descripcion
 - dimensiones
 - estado_activo
 - fecha_actualizacion
 - info_tallas
 - nombre
 - peso
 - precio_unitario
 - precio_venta
 - url_imagen
 - Indexes
 - Foreign Keys
 - Triggers
 - Views
 - Stored Procedures
 - Functions
 - db_iquadra
 - edteam
 - filament
 - filament_laravel
 - gestionale-libreria

producto_entity db_footsy_java producto_entity x

Info Columns Indexes Triggers Foreign keys Partitions Grants DDL

Indexes in Table

Visible	Key	Type	Uni...	Columns
☒	PRIMARY	BTREE	YES	id
☒	UKlwkiuffesrtnpdnr...	BTREE	YES	codigo

Index Details

Key Name:
Index Type:
Allows NULL:
Cardinality:
Comment:
User Comment:

Columns in table

Column	Type	Nullable	Indexes
id	int	NO	PRIMARY
codigo	varchar(255)	NO	UKlwkiuffesrtnpdnr8th4bp7
descripcion	varchar(255)	YES	
dimensiones	varchar(255)	YES	
estado_activo	bit(1)	YES	
fecha_actualizacion	datetime(6)	YES	
fecha_creacion	datetime(6)	YES	
info_tallas	enum('EU_35','EU_36','EU...	YES	
nombre	varchar(255)	YES	
peso	varchar(255)	YES	
precio_unitario	double	YES	
precio_venta	double	YES	
url_imagen	varchar(255)	YES	

Pasos de la actividad

Paso 1. Configuración inicial del proyecto

Como primer paso se creó la estructura base del proyecto utilizando Spring Initializr, seleccionando las siguientes configuraciones:

Project: Maven
Language: Java
Spring Boot: 4.0.7
Java: 17

Posteriormente se agregaron las dependencias necesarias para el desarrollo del proyecto:

- Spring Web
- Spring Data JPA
- Lombok
- MySQL Driver
- Thymeleaf

Una vez descargado el proyecto generado por Spring Initializr, se descomprimió y se abrió en IntelliJ IDEA para iniciar el desarrollo.

Finalmente, en MySQL Workbench se creó la base de datos **db_footsy_java**, la cual será utilizada durante el desarrollo del módulo Producto.

Paso 2. Configuración del proyecto y creación del módulo Producto

Con el proyecto creado, se configuró el archivo **application.properties**, ubicado en la carpeta resources, estableciendo los parámetros necesarios para la conexión con la base de datos:

- Nombre de la aplicación.
- URL de conexión.
- Usuario.
- Contraseña.
- Driver JDBC de MySQL.
- Configuración de Hibernate para la creación y actualización automática de las tablas.

Una vez establecida la conexión, se creó la entidad **ProductoEntity**, definiendo sus atributos y configurándola mediante las anotaciones de JPA para que Hibernate la reconociera como una entidad persistente.

Utilizamos Lombok para los getter, setter y constructores.

Paso 3. Creacion de HomeController y vista Index.html

Posteriormente se creó la clase **HomeController**, donde se definieron las rutas iniciales del módulo Producto.

```
@Controller
public class HomeController {
    @GetMapping("/")
    public String index(){
        return "index";
    }
}
```

Finalmente, dentro de la carpeta resources/**resources**, se creó la vista **index.html**, que será utilizada para visualizar la información del módulo Producto.

Dentro de la carpeta resources/static se creó la carpeta css con el archivo style.css, donde se definieron los estilos de la aplicación. Posteriormente, este archivo se vinculó con la página index.html mediante la etiqueta <link> en la sección <head>.

```
<!-- CSS -->
<link rel="stylesheet" th:href="@{/css/style.css}">
</head>
```

Paso 4. Creación del repositorio del módulo Producto

Una vez creada la entidad **ProductoEntity**, el siguiente paso consiste en desarrollar el acceso a los datos mediante un **Repository**.

Para ello se crea una nueva Java Class pero en vez de clase es una **interface**. En Spring Data JPA, un repositorio es una interfaz que gestiona la comunicación entre la aplicación y la base de datos. Hibernate implementa automáticamente las operaciones básicas de persistencia sin que sea necesario escribir instrucciones en SQL de forma manual.

Al interior de la interfaz **ProductoRepository** se utiliza **extends JpaRepository<ProductoEntity, Long>**. Esto aplica el concepto de herencia de la Programación Orientada a Objetos (POO), ya que **ProductoRepository** hereda todos los métodos proporcionados por JpaRepository, como save(), findAll(), findById() y deleteById(). Además, al indicar **ProductoEntity** se establece que el repositorio gestionará la entidad **ProductoEntity**, mientras que **Integer** representa el tipo de dato de su clave primaria (ID).

```
public class ProductoEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
```

```
1 package com.footsy.demo.repository;
2
3
4 import com.footsy.demo.entity.ProductoEntity;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7
8 @Repository no usages
9 public interface ProductoRepository extends JpaRepository<ProductoEntity, Integer> {
10
11 }
12
```

Paso 5. Creación del Services (ProductServices)

El objetivo de services es separar la lógica del negocio del controlador. Así el controlador recibe la petición del usuario y delega el trabajo al servicio.

Services actúa como intermediaria entre el controlador y el repositorio, permitiendo mantener una mejor organización del proyecto y respetando la arquitectura de SpringBoot, es la capa encargada de implementar la lógica del negocio, coordinar las operaciones necesarias antes de acceder a la base de datos.

Dentro de la carpeta Services creamos la clase **ProductoService** y agregamos **@Services** y **@Autowired**

```
1 package com.footsy.demo.service;
2
3 import com.footsy.demo.repository.ProductoRepository;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.stereotype.Service;
6
7 @Service
8 public class ProductoService {
9
10     @Autowired
11     private ProductoRepository productoRepository;
12 }
```

La anotación **@Service** le indica a SpringBoot que la clase pertenece a la capa de servicio. La anotación **@Autowired** realiza la inyección de dependencias. En lugar de crear manualmente un objeto de ProductoRepository, spring lo crea automáticamente y lo asigna a la

variable. Como ProductRepository es una interfaz, no puede instanciarse directamente, spring detecta la interfaz, genera una implementación automáticamente y la inyecta en la clase ProductoServices,

```
@Service
public class ProductoService {

    @Autowired
    private ProductRepository productoRepository;
}
```

Private ProductRepository productoRepository, es una variable que nos permitirá a acceder a todos los método que ofrece JpaRepository, como Save,find, findById, deleteById

Paso 6. Implementación método guardarProducto() - CREATE

En la clase ProductoService se creó el método guardarProducto(), encargado de recibir un objeto ProductoEntity y delegar al repositorio la operación de almacenamiento mediante el método save()

```
/*Crear un nuevo producto*/
public ProductoEntity guardarProducto(ProductoEntity productoEntity) {
    return productoRepository.save(productoEntity);
}
```

Paso 7. Implementación método obtenerTodosLosProductos() -READ

En la clase ProductoService se creó el método obtenerTodosLosProductos(), encargado de consultar todos los productos registrados en la base de datos.

```
/*ObtenerTodosLosProductos*/
public List<ProductoEntity> obtenerTodosLosProductos() {
    return productoRepository.findAll();
}
```

Paso 8. Implementación método obtenerProductoPorId()

En la clase ProductoService se creó el método obtenerProductoPorId(), que permitirá buscar un producto específico a partir de su identificador ID

```

/*ObtenerProductoId*/
public Optional<ProductoEntity> obtenerProductoPorId(Integer id) {
    return productoRepository.findById(id);
}

```

Paso 9. Implementación método actualizarProducto() - UPDATE

En la clase ProductoService se creó el método actualizarProducto (), que permitirá actualizar la información de un producto existente en la base de datos.

```

/*actualizar producto*/
public ProductoEntity actualizarProducto(Integer id, ProductoEntity productoActualizado) {
    Optional<ProductoEntity> productoExistente = productoRepository.findById(id);
    if (productoExistente.isPresent()) {
        productoActualizado.setId(id);
        return productoRepository.save(productoActualizado);
    }
    return null;
}

```

Paso 10. Implementación método eliminarProducto() - DELETE

En la clase ProductoService se creó el método eliminarProducto(), que permitirá eliminar un producto de la base de datos utilizando su identificador.

Antes de realizar la eliminación, se verifica que el producto exista para evitar errores al intentar eliminar un registro inexistente.

```

/*Delete*/
public boolean eliminarProducto(Integer id) { no usages
    Optional<ProductoEntity> productoEntity = productoRepository.findById(id);
    if (productoEntity.isPresent()) {
        productoRepository.deleteById(id);
        return true;
    }
    return false;
}

```

Paso 11. Creación de ProductoController

Ahora se crea la clase **ProductoController**, que se encargara de recibir las solicitudes realizadas por el usuario desde el navegador, comunicarse con la capa de servicio y devolver la vista correspondiente.

```

@Controller
@RequestMapping(path = "/productos")
public class ProductoController {

    @Autowired
    private ProductoService productoService;
}

```

La anotación **@Controller** indica a hibernate que es un controlador, que será la encargada de recibir las peticiones HTTP que llegan desde el navegador. **@RequestMapping("/productos")** esta anotación define la ruta base del controlador. **@Autowired** ayuda a Spring creando automáticamente un objeto de **ProductoService** así el controlador puede utilizar todos los métodos que fueron implementados en **ProductoService**.

- guardarProducto()
- obtenerTodosLosProductos()
- obtenerProductoPorId()
- actualizarProducto()
- eliminarProducto()

Paso 12. Implementación en ProductoController del método mostrarPaginaProducto()

Se procede con la creación del primer método del controlador que permite atender la solicitud del usuario consultar la lista de productos y enviarla a la vista Index.html para su visualización,

```

@GetMapping
public String mostrarPaginaProductos(Model model) {
    model.addAttribute(attributeName: "productos",
        productoService.obtenerTodosLosProductos());
    return "index";
}

```

La anotación **@GetMapping** indica que el método responderá a las peticiones HTTP GET

Paso 13. Implementación de Index.html

La página **index.html** ya contaba con una estructura base. sin embargo, para visualizar los productos almacenados en la base de datos fue necesario implementar una **card dinámica** que, mediante la iteración de la colección

de productos enviada desde el controlador, mostrara la información de cada registro (nombre, imagen, precio, descripción, entre otros).

Como la base de datos inicialmente no contenía información, se realizaron varias instrucciones SQL INSERT para cargar datos de prueba. Esto permitió verificar el correcto funcionamiento de la consulta, la integración con Spring Boot y el renderizado dinámico de los productos en la vista utilizando Thymeleaf.

Ahora cuando se ingresa a **localhost:8080/productos** se ve la página, con los productos que se ingresaron en la base de datos.

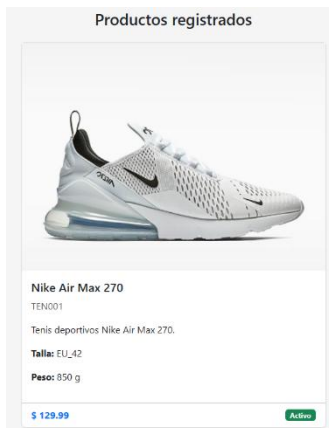
INSERT SQL en la Tabla Producto

```
INSERT INTO producto_entity (
    estado_activo,
    fecha_creacion,
    fecha_actualizacion,
    codigo,
    nombre,
    descripcion,
    precio_unitario,
    precio_venta,
    info_tallas,
    peso,
    dimensiones,
    url_imagen
) VALUES
(TRUE, NOW(), NOW(), 'TEN001', 'Nike Air Max 270', 'Tenis deportivos Nike Air Max 270.', 95.00, 129.99,
'EU_42', '850 g', '32x20x12 cm', 'nike_airmax270.jpg'),
(TRUE, NOW(), NOW(), 'TEN002', 'Adidas Ultraboost Light', 'Tenis para running Adidas Ultraboost.',
110.00, 149.99, 'EU_41', '820 g', '32x20x12 cm', 'adidas_ultraboost.jpg'),
(TRUE, NOW(), NOW(), 'TEN003', 'Puma RS-X', 'Tenis urbanos Puma RS-X.', 80.00, 119.99, 'EU_39', '810 g',
'32x20x12 cm', 'puma_rsx.jpg'),
(TRUE, NOW(), NOW(), 'TEN004', 'Reebok Classic Leather', 'Modelo clásico Reebok.', 70.00, 99.99,
'EU_40', '780 g', '31x20x12 cm', 'reebok_classic.jpg'),
(TRUE, NOW(), NOW(), 'TEN005', 'New Balance 574', 'Tenis casuales New Balance.', 75.00, 109.99,
'EU_43', '790 g', '31x20x12 cm', 'nb_574.jpg'),
(TRUE, NOW(), NOW(), 'TEN006', 'Asics Gel Nimbus 26', 'Tenis para entrenamiento.', 120.00, 169.99,
'EU_44', '830 g', '33x20x12 cm', 'asics_nimbus26.jpg'),
(TRUE, NOW(), NOW(), 'TEN007', 'Converse Chuck Taylor', 'Tenis clásicos Converse.', 50.00, 79.99,
'EU_38', '760 g', '31x20x12 cm', 'converse_chuck.jpg'),
(TRUE, NOW(), NOW(), 'TEN008', 'Vans Old Skool', 'Tenis casuales Vans.', 55.00, 84.99, 'EU_41', '770 g',
'31x20x12 cm', 'vans_oldskool.jpg'),
(TRUE, NOW(), NOW(), 'TEN009', 'Under Armour Charged', 'Tenis deportivos Under Armour.', 85.00,
119.99, 'EU_45', '820 g', '32x20x12 cm', 'ua_charged.jpg'),
(TRUE, NOW(), NOW(), 'TEN010', 'Fila Disruptor II', 'Modelo urbano Fila.', 65.00, 94.99, 'EU_37', '840 g',
'32x20x12 cm', 'fila_disruptor.jpg'),
(TRUE, NOW(), NOW(), 'TEN011', 'Skechers Go Walk', 'Tenis cómodos para caminar.', 60.00, 89.99,
'EU_40', '700 g', '30x19x11 cm', 'skechers_gowalk.jpg'),
(TRUE, NOW(), NOW(), 'TEN012', 'Mizuno Wave Rider', 'Tenis para corredores.', 110.00, 149.99, 'EU_42',
'810 g', '32x20x12 cm', 'mizuno_wave.jpg'),
(TRUE, NOW(), NOW(), 'TEN013', 'Jordan Max Aura', 'Tenis Jordan para basketball.', 115.00, 159.99,
'EU_44', '850 g', '33x20x12 cm', 'jordan_aura.jpg'),
```

```
(TRUE, NOW(), NOW(), 'TEN014', 'Nike Revolution 7', 'Modelo ligero Nike.', 60.00, 89.99, 'EU_39', '760 g',
'31x20x12 cm', 'nike_revolution7.jpg'),
(TRUE, NOW(), NOW(), 'TEN015', 'Adidas Forum Low', 'Tenis casuales Adidas.', 90.00, 129.99, 'EU_43', '810
g', '32x20x12 cm', 'adidas_forum.jpg'),
(TRUE, NOW(), NOW(), 'TEN016', 'Puma Suede Classic', 'Modelo clásico Puma.', 70.00, 99.99, 'EU_36',
'790 g', '31x20x12 cm', 'puma_suede.jpg'),
(TRUE, NOW(), NOW(), 'TEN017', 'Reebok Nano X4', 'Tenis para entrenamiento funcional.', 105.00, 145.99,
'EU_45', '820 g', '32x20x12 cm', 'reebok_nano.jpg'),
(TRUE, NOW(), NOW(), 'TEN018', 'New Balance Fresh Foam X', 'Máxima comodidad para correr.', 125.00,
169.99, 'EU_46', '830 g', '33x20x12 cm', 'nb_freshfoam.jpg'),
(TRUE, NOW(), NOW(), 'TEN019', 'On Cloud 5', 'Tenis ligeros de alto rendimiento.', 130.00, 179.99, 'EU_41',
'740 g', '31x20x12 cm', 'on_cloud5.jpg'),
(TRUE, NOW(), NOW(), 'TEN020', 'Salomon Speedcross 6', 'Tenis para trail running.', 135.00, 189.99,
'EU_42', '860 g', '33x21x12 cm', 'salomon_speedcross.jpg');
```

Card Dinamica

```
<!-- PRODUCTOS -->
<div class="container py-5">
  <h2 class="mb-4 text-center">
    Productos registrados
  </h2>
  <div class="row g-4">
    <div class="col-md-4 col-lg-3"
      th:each="producto : ${productos}">
      <div class="card h-100">
        
        <div class="card-body">
          <h5 class="card-title"
            th:text="${producto.nombre}">
            Nombre
          </h5>
          <p class="text-muted"
            th:text="${producto.codigo}">
            Código
          </p>
          <p class="card-text"
            th:text="${producto.descripcion}">
            Descripción
          </p>
          <p>
            <strong>Talla:</strong>
            <span th:text="${producto.infoTallas}">
              Talla
            </span>
          </p>
          <p>
            <strong>Peso:</strong>
            <span th:text="${producto.peso}">
              Peso
            </span>
          </p>
        </div>
        <div class="card-footer bg-white">
          <div class="d-flex justify-content-between align-items-center">
            <span class="fw-bold text-primary"
              th:text="'$ ' + ${producto.precioVenta}">
              $0
            </span>
            <span class="badge bg-success"
              th:text="${producto.estadoActivo} ? 'Activo' : 'Inactivo'">
              Activo
            </span>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
```



La card tiene la información de

```

${producto.urlImagen}}
${producto.nombre}}
${producto.codigo}}
${producto.descripcion}}
${producto.infoTallas}}
${producto.peso}}
${producto.precioVenta}}

${producto.estadoActivo}}

```

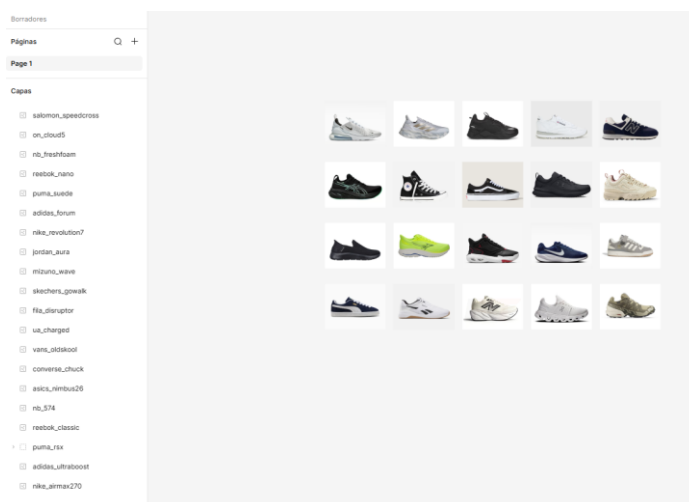
Paso 14. Integrar imágenes en el Proyecto

Con el fin de visualizar las imágenes de los productos dentro de la aplicación, se creó la carpeta `img` dentro de `src/main/resources/static`, que es el directorio utilizado por Spring Boot para almacenar recursos estáticos como imágenes, hojas de estilo y archivos JavaScript.

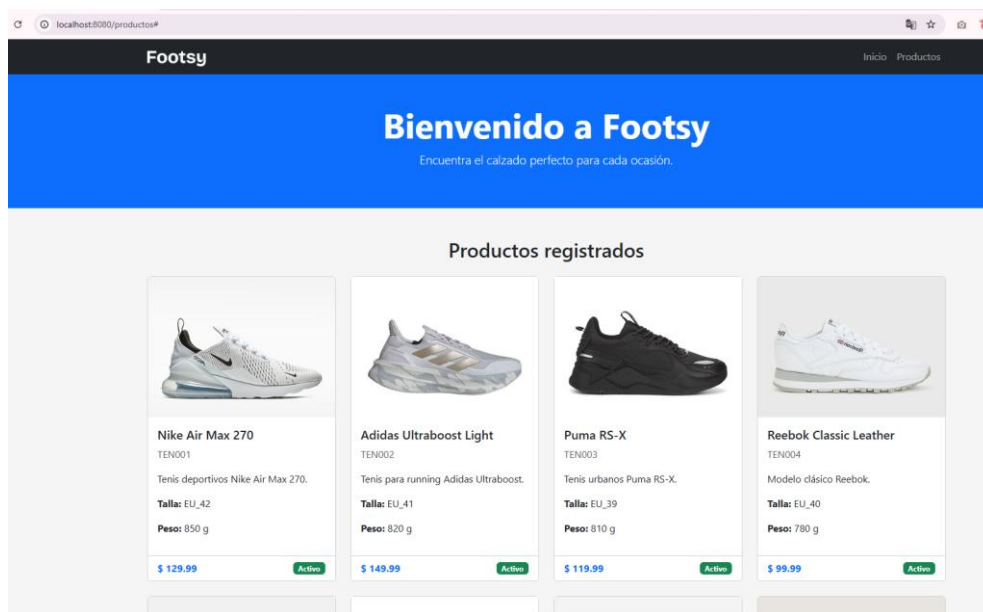
Posteriormente, se copiaron las imágenes de los productos dentro de esta carpeta, respetando exactamente el nombre de cada archivo registrado previamente en la base de datos durante la inserción de los datos (INSERT).

De esta manera, cuando la aplicación consulta la información del producto, utiliza el nombre almacenado en la base de datos para localizar automáticamente la imagen correspondiente dentro de la carpeta `img` y mostrarla en la interfaz de usuario.

Se preparan las imágenes en **figma** para que tengan el mismo tamaño

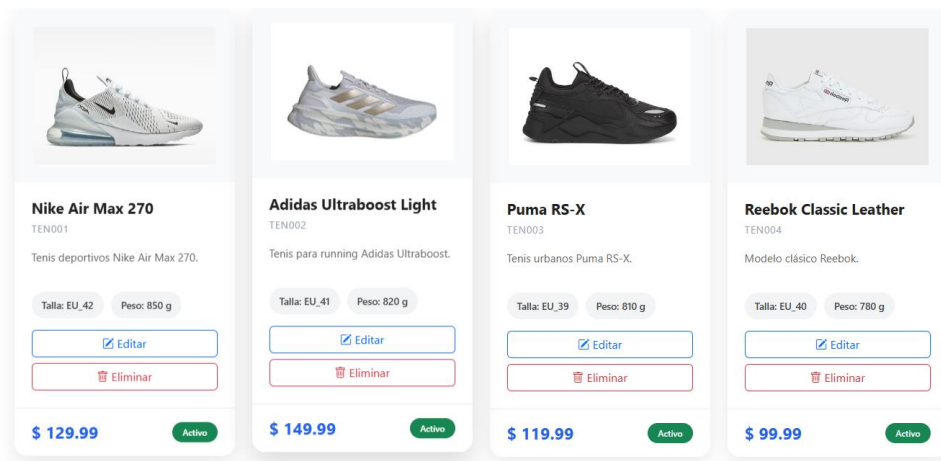


Ahora se ve la pagina asi:



Paso 15. Integración del crear producto en la Hero

Se procedió a mejorar los estilos y integrar en el menú dinámico los botones de editar y eliminar.



En la HERO de la página, se creó un botón “crea producto” para agregar un nuevo producto a la base de datos.

Paso 16. Creación del formulario Producto (nuevo.html)

Dentro de la carpeta templates/producto se creó la vista nuevo.html, la cual contiene un formulario desarrollado con Bootstrap para facilitar el ingreso de la información del producto.

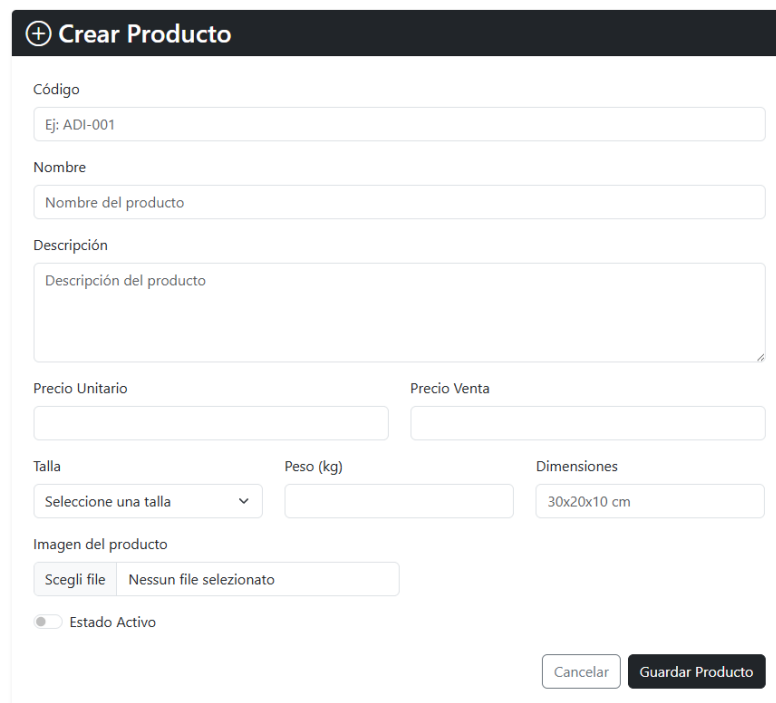
El formulario incorpora los campos definidos en la entidad ProductoEntity, permitiendo registrar la información que posteriormente será almacenada en la base de datos.

- Importante que el formulario punte a productos/guardar

```
<form th:action="@{/productos/guardar}"
      th:object="${producto}"
      method="post"
      enctype="multipart/form-data">
```

Los campos incluidos son:

- Código
- Nombre
- Descripción
- Precio Unitario
- Precio Venta
- Talla
- Peso
- Dimensiones
- URL Imagen
- Estado Activo



Paso 17. Creación de la ruta para mostrar el formulario en nuevo.html

Dentro de **ProductoController** se agrego un nuevo método que nos permitirá mostrar dicho formulario. (mostrarFormularioNuevoProducto), Este nuevo método permitirá que cuando hagamos localhost:8080/productos y hagamos click en el botón “crea producto” nos lleve a localhost:8080/productos/nuevo

```

R Rename usages
@GetMapping("/nuevo")
public String mostrarFormularioNuevoProducto(Model model){
    model.addAttribute(attributeName: "producto", new ProductoEntity());
    return "producto/nuevo";
}

```

New ProductoEntity() es la creación de un objeto vacío que tendrá la información que vamos a ingresar. (se prepara un objeto para que thymeleaf pueda asociar los campos del form.).

La notación **@GetMapping**, hace referencia que haremos una petición GET, porque solo se leerá información.

Paso 18. Metodo guardarProducto()

En **nuevo.html**, cuando se completa la información del nuevo producto al hacer guardar producto, se busca que la información se almacene en la base de datos. Para ello, es importante implementar el método **guardarProducto**. Este método no será @GetMapping sino será un post, ósea **@PostMapping** porque recibe el formulario y guarda la información del producto.

```

/* cuando envía el formulario de nuevo.html salva en la DB*/
@PostMapping("/crear")
public String guardarProducto(@ModelAttribute ProductoEntity productoEntity){
    productoService.guardarProducto(productoEntity); /* la referencia al metodo services*/
    return "redirect:/productos";
}

```

Paso 19. Validación de funcionamiento de nuevo.html

Haciendo una validación, La información no se guarda en la base de datos, el método **guardarProducto** no salva la información de la imagen, para eso es necesario modificarlo. Así como tomar la imagen y copiarla en resources.

```

@PostMapping("/guardar")
public String guardarProducto
    (@ModelAttribute ProductoEntity productoEntity, // la info del form
    @RequestParam("imagen") MultipartFile imagen){ // la info de la imagen
    //obtenemos el nombre de la imagen
    String nombreArchivo = imagen.getOriginalFilename();
    // Crear la ruta donde se guardará la imagen
    Path ruta = Paths.get("src/main/resources/static/img/" + nombreArchivo);
    // Copiar la imagen a la carpeta img
    Files.copy(imagen.getInputStream(), ruta);
    //guardar el nombre de la imagen en producto
    productoEntity.setUrlImagen(nombreArchivo);
    //guardar toda la informacion en la bd
    productoService.guardarProducto(productoEntity); /* la referencia al metodo services*/
    return "redirect:/productos";
}

```

La funcionalidad CREATE esta completada. La anotación `@ModelAttribute` Recibe todos los campos del formulario y spring crea automáticamente `ProductoEntity` mientras `@RequestParam` recibe únicamente el archivo seleccionado en este caso la imagen. Asignamos a una variable el valor de `imagen.getOriginalFilename()`, ósea el nombre del archivo.

```
/* cuando envia el formulario de nuevo.html salva en la DB*/
@PostMapping("/guardar")
public String guardarProducto
(@ModelAttribute ProductoEntity productoEntity, // la info del form
@RequestParam("imagen") MultipartFile imagen) throws IOException { // la info de la imagen
    if (!imagen.isEmpty()) {
        String nombreArchivo = imagen.getOriginalFilename();
        Path ruta = Paths.get("src/main/resources/static/img/" + nombreArchivo);
        Files.copy(imagen.getInputStream(), ruta,
            StandardCopyOption.REPLACE_EXISTING);
        productoEntity.setUrlImagen(nombreArchivo);
    } else if (productoEntity.getId() != null) {
        ProductoEntity productoExistente = productoService
            .obtenerProductoPorId(productoEntity.getId())
            .orElseThrow(() -> new RuntimeException("Producto no encontrado"));
        productoEntity.setUrlImagen(productoExistente.getUrlImagen());
    }
    productoService.guardarProducto(productoEntity);
    return "redirect:/productos";
}
```

Paso 20. Implementación del método editarProducto()

El objetivo es que cuando un usuario seleccione un producto existente pueda modificar su información. Para este paso se reutilizo el mismo formulario de nuevo.html, ahí se cargaron automáticamente los datos del producto seleccionado.

Para ello, en index.html, en el interior de la card se agrego un botón “Editar”

```
<div class="d-grid gap-2 mt-3">
    <a th:href="@{/productos/editar/{id}(id=${producto.id})}"
        class="btn btn-outline-primary">
        <i class="bi bi-pencil-square"></i>
        Editar
    </a>
```

Ahora, cada Card dinámica tiene un Id, la idea de este método es que reciba el id seleccionado, lo busque en la base de datos y envíe esa información al formulario para que el usuario pueda modificarla.

Dentro de **ProductoController** se agregó el método `editarProducto()`, encargado de recuperar la información de un producto para permitir su modificación. Para ello se utilizó la anotación `@PathVariable`, la cual permite a Spring obtener automáticamente el valor del identificador (id) enviado en la URL. Posteriormente, se invocó el método correspondiente del servicio

(ProductoService), el cual busca el producto asociado a ese identificador y devuelve la entidad completa.

Como se trata de un producto ya existente que será modificado, no se crea una nueva instancia del objeto utilizando `new ProductoEntity()`, como ocurre en el proceso de creación. En su lugar, se envía al modelo el objeto recuperado de la base de datos mediante la instrucción:

```
@GetMapping("/editar/{id}")
public String editarProducto(@PathVariable Integer id, Model model) {
    ProductoEntity producto = productoService
        .obtenerProductoPorId(id)
        .orElseThrow(() -> new RuntimeException("Producto no encontrado"));
    model.addAttribute("producto", producto);
    return "producto/nuevo";
}
```

`.orElseThrow(() -> new RuntimeException("Producto no encontrado"));`, Significa si el producto existe, devolverlo. Si no existe, entonces lance una excepción.

Paso 21. Implementación del método eliminarProducto()

El objetivo es que cuando un usuario seleccione un producto existente pueda eliminarlo. Lo primero fue crear en index.html en la card dinámica el botón de “Eliminar”

```
<a th:href="@{/productos/eliminar/{id}(id=${producto.id})}"
    class="btn btn-outline-danger"
    onclick="return confirm('¿Deseas eliminar este producto?');">
    <i class="bi bi-trash"></i>
    Eliminar
</a>
```

Dentro de **ProductoController** se agregó el método “eliminarProducto”, encargado de eliminar un producto de la base de datos. Para ello se utilizó la anotación `@PathVariable`, que permite a Spring obtener automáticamente el valor del identificador (id) enviado en la URL. Posteriormente, se invocó el método correspondiente del servicio (ProductoService), el cual se encarga de localizar y eliminar el producto asociado a ese identificador. Finalmente, se redirecciona nuevamente al listado de productos para visualizar los cambios realizados.

```
//DELETE ELIMINA PRODUCTO DEL DATABASE
@GetMapping("/eliminar/{id}")
public String eliminarProducto(@PathVariable Integer id) {
    productoService.eliminarProducto(id);
    return "redirect:/productos";
}
```

Paso 22. Implementación del método `mostrarDetalleProducto()`

Dentro de **ProductoController** se agregó el método `mostrarDetalleProducto()`, encargado de visualizar la información completa de un producto seleccionado. Para ello se utilizó la anotación `@PathVariable`, la cual permite a Spring obtener automáticamente el valor del identificador (id) enviado en la URL. Posteriormente, se invocó el método correspondiente del servicio (**ProductoService**), el cual busca el producto asociado a ese identificador y devuelve la entidad completa.

Una vez recuperada la información, el producto se envía al modelo mediante la instrucción `model.addAttribute()`. De esta manera, la vista `detalle.html` recibe el objeto y puede mostrar toda la información del producto, como el código, nombre, descripción, precio, talla, peso, dimensiones, estado e imagen.

```
//LISTAR PRODUCTOS POR ID //profe ListarProducto
@GetMapping("/{detalle/{id}}")
public String mostrarDetalleProducto(@PathVariable Integer id, Model model){
    ProductoEntity producto = productoService
        .obtenerProductoPorId(id)
        .orElseThrow(() -> new RuntimeException("Producto no encontrado"));
    model.addAttribute(attributeName: "producto", producto);
    return "producto/detalle";
}
```

Para permitir la visualización del detalle de cada producto, se agregó en la card dinámica de `index.html` un nuevo botón denominado "Ver detalle", el cual redirecciona al usuario hacia la vista correspondiente al producto seleccionado mediante el envío de su identificador (id).

Posteriormente, se creó en la carpeta `templates/producto` la vista `detalle.html`. Al seleccionar el botón "Ver detalle", Spring recibe el identificador del producto a través de la URL, ejecuta el método `mostrarDetalleProducto()` del **ProductoController**, recupera la información desde la base de datos mediante el servicio y finalmente la envía a la vista `detalle.html` para su visualización.

```
<a th:href="@{/productos/detalle/{id}(id=${producto.id})}"
    class="btn btn-dark w-100">
    <i class="bi bi-eye"></i>
    Ver detalle
</a>
```

Git

Durante el desarrollo del módulo de gestión de productos se utilizó Git como sistema de control de versiones, permitiendo registrar y organizar cada uno de los avances realizados en la aplicación. Gracias a esta herramienta fue posible mantener un historial de cambios, facilitando la gestión del código fuente y la recuperación de versiones anteriores en caso de ser necesario.

Repositorio Git del proyecto

Actualmente el proyecto se encuentra alojado en la organización ADSO Learning Hub de GitHub, correspondiente a las actividades de formación del programa ADSO. El repositorio utilizado para esta evidencia es:

https://github.com/ADSO-SENA-GARCIAPATRICIA2025/CRUD_Productos_Java-GA7-220501096-AA2-EV01

Estándares de Codificación

Durante el desarrollo del proyecto se aplicaron buenas prácticas de programación con el fin de obtener un código organizado, legible y fácil de mantener.

Entre los estándares aplicados se encuentran:

- Organización del proyecto siguiendo la arquitectura MVC (Model, View y Controller).
- Separación de responsabilidades entre las capas Controller, Service, Repository y Entity.
- Uso de convenciones de nomenclatura en clases, métodos y atributos utilizando PascalCase para las clases y camelCase para variables y métodos.
- Uso de anotaciones propias de Spring Boot como @Controller, @Service, @Repository, @Entity, @GetMapping, @PostMapping y @Autowired.
- Implementación de Thymeleaf para la generación dinámica de las vistas.
- Uso de Hibernate (JPA) para la persistencia de datos y el acceso a la base de datos.
- Organización de los recursos estáticos (CSS e imágenes) dentro de la carpeta resources/static.

CONCLUSIONES

El desarrollo de esta evidencia me permitió aprender sobretodo hibernate, comprender como implementar el CRUD en el modulo productos utilizando SpringBoot, hibernate, thymeleaf y mysql. durante el desarrollo se presentaron diferentes errores relacionados con thymeleaf, hibernate y la gestión de las imágenes, los cuales me permitieron fortalecer la comprensión del funcionamiento interno de springboot y mejorar las habilidades para la solución de problemas durante el desarrollo de aplicaciones web.

REFERENCIAS

- *Servicio Nacional de Aprendizaje (SENA). (s.f.). Objeto virtual de aprendizaje (OVA) – CF18. Repositorio Institucional SENA. Recuperado el 12 de junio de 2026, de https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118_1/Contenido/OVA/CF18/index.html*
- *Servicio Nacional de Aprendizaje (SENA). (s.f.). Objeto virtual de aprendizaje (OVA) – CF19. Repositorio Institucional SENA. Recuperado el 12 de junio de 2026, de https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118_1/Contenido/OVA/CF19/index.html*
- *Spring. (s.f.). Spring Boot Reference Documentation. Recuperado el 2 de agosto de 2026, de <https://docs.spring.io/spring-boot/documentation.html>*
- *Spring. (s.f.). Spring Framework Documentation. Recuperado el 2 de agosto de 2026, de <https://docs.spring.io/spring-framework/reference/>*
- *Hibernate. (s.f.). Hibernate ORM Documentation. Recuperado el 2 de agosto de 2026, de <https://hibernate.org/orm/documentation/>*
- *Thymeleaf. (s.f.). Thymeleaf Documentation. Recuperado el 2 de agosto de 2026, de <https://www.thymeleaf.org/documentation.html>*